AWS NOTES

From Beginner to Advanced

1. AWS FUNDAMENTALS & GLOBAL INFRASTRUCTURE

☆ What is AWS?

AWS (Amazon Web Services) is a cloud platform that provides 200+ fully featured services from data centers worldwide.

☆ Why Cloud?

- No upfront hardware cost
- Pay-as-you-go
- Highly scalable
- High availability & reliability
- Global reach
- Focus on business, not infrastructure

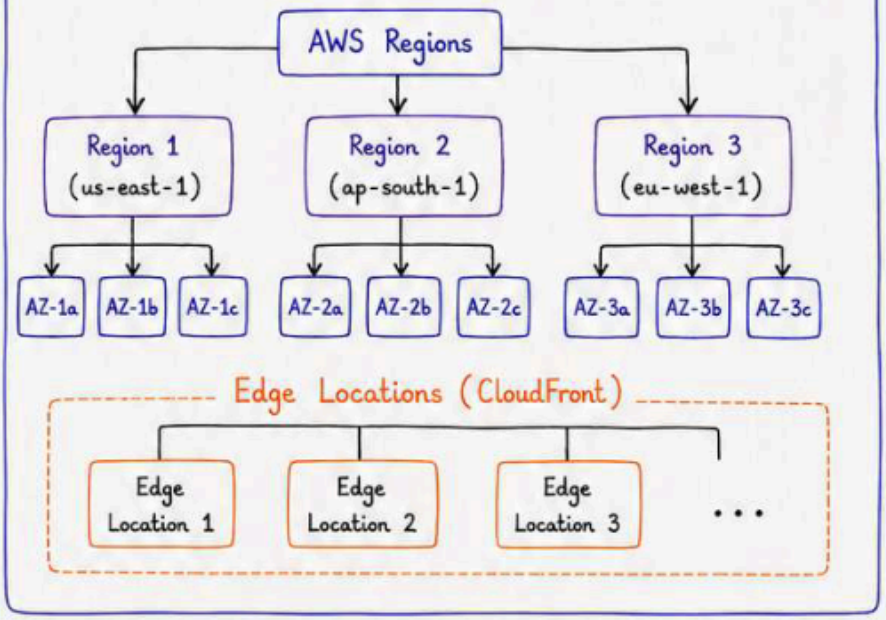
☆ IaaS vs PaaS vs SaaS

Model	You Manage	AWS Manages	Examples
IaaS	OS, Applications, Runtime, Data, Middleware	Servers, Storage, Networking, Virtualization	EC2, VPC, S3, EBS
PaaS	Applications, Data	OS, Runtime, Middleware, Servers, Storage, Networking	Elastic Beanstalk, RDS, Lambda
SaaS	Data, Settings	Applications, Runtime, OS, Servers, Storage, Networking	S3, DynamoDB, Google Workspace

☆ AWS Global Infrastructure

- Region : A physical location in the world (e.g., ap-south-1 - Mumbai).
- Availability Zone (AZ) : One or more isolated data centers within a Region.
- Edge Location : Used by CloudFront to cache content closer to users.
- Local Zone : Extends a Region with infrastructure closer to large population centers.

☆ AWS Global Infrastructure (Overview)



☆ Shared Responsibility Model

AWS Responsibility	Customer Responsibility
• Physical Infrastructure • Compute • Storage • Database • Networking • AWS Global Infrastructure	• Data • Applications • OS, Middleware • Access Management • Security Groups • Monitoring & Logging

☆ How AWS Works?

Access (AWS Console / CLI / SDK (User Interface)) → Service (e.g., EC2, S3, RDS, Lambda) → Resource (Instance, Bucket, DB, Function) → Action / Operation (Create, Read, Update, Delete) → Result (Your Application Runs in Cloud)

2. COMPUTE SERVICES

Compute services let you run your applications in the cloud.

☆ 1. EC2 (Elastic Compute Cloud)

Virtual servers in the cloud.

- Full control over OS, updates, security
- Pay for what you use
- Stop / Start / Terminate as needed

☆ 2. AMI (Amazon Machine Image)

A template used to launch EC2 instances.

- Contains OS, software & configuration
- You can create custom AMIs

☆ 3. Instance Types

Different types for different needs.

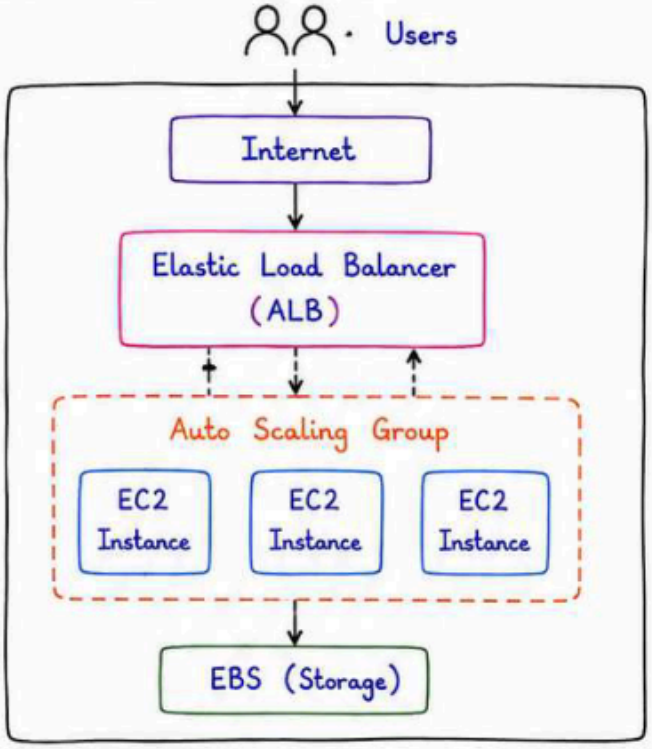
- General Purpose : t3, t2
- Compute Optimized : c5, c6
- Memory Optimized : r5, x1e
- Storage Optimized : i3, d2

☆ 4. Auto Scaling Group (ASG)

Automatically adjusts capacity based on traffic and demand.

- Scale Out → Add more instances
- Scale In → Remove extra instances
- Improves availability & saves cost

5. Architecture (High Level)



6. Launch Template

A template that contains configuration details required to launch EC2 instances.

- AMI ID
- Instance Type
- Security Group
- Key Pair, User Data, etc.

7. Security Group (Basic)

Acts as a virtual firewall for your instance.

- Works at instance level
- Stateful → Return traffic is allowed
- Allow rules (Inbound) & Outbound rules

8. Elastic Load Balancer (ELB)

Distributes incoming traffic across multiple targets (instances) to improve availability.

- Types : ALB (Layer 7)
  NLB (Layer 4)
- Health checks ensure only healthy instances receive traffic.

EC2 Instance Purchasing Options

Option	Description	Use Case
On-Demand	Pay by the hour/second. No long term commitment.	Short term, unpredictable workloads
Reserved Instance	Pay upfront for 1 or 3 years. Discounted.	Steady, long term workloads
Spot Instance	Bid for spare EC2 capacity. Very low cost.	Fault tolerant, flexible workloads
Savings Plan	Commit to a certain $ per hour usage.	Flexible & cost effective

3. STORAGE SERVICES

Storage services help you store, protect and retrieve data in the cloud.

1. S3 (Simple Storage Service)

- Object storage service to store any amount of data.
- Highly durable, secure and scalable.
- Access data from anywhere on the web.

S3 Use Cases

- Backup & Restore
- Static Website
- Data Lakes
- Big Data Analytics
- Media Hosting

2. S3 Storage Classes

- Standard : Frequent access data
- Intelligent-Tiering : Automatic cost optimization
- Standard-IA : Infrequent access
- One Zone-IA : Infrequent, single AZ
- Glacier Instant Retrieval : Archive, millisecond access
- Glacier Flexible Retrieval : Archive, minutes to hours
- Glacier Deep Archive : Archive, hours to days (lowest cost)

3. S3 Key Features

- ✓ Unlimited storage
- ✓ 99.999999999% (11 9's) durability
- ✓ Versioning
- ✓ Lifecycle Rules
- ✓ Encryption (SSE-S3, SSE-KMS)
- ✓ Bucket Policies & Access Control
- ✓ Cross-Region Replication
- ✓ Event Notifications

4. EBS (Elastic Block Store)

- Block storage for EC2 instances.
- Data is tied to a specific AZ.
- Low-latency, high performance.
- Types :
  - gp3 (General Purpose SSD)
  - io2 (Provisioned IOPS SSD)
  - st1 (Throughput Optimized HDD)
  - sc1 (Cold HDD)

5. EFS (Elastic File System)

- Managed NFS file system.
- Can be shared across multiple EC2 instances.
- Scalable, elastic and Highly Available.
- Use case : Web servers, content management, home directories.

6. FSx (Fully Managed Storage)

- Fully managed third-party file systems.
- Types :
  - FSx for Windows File Server
  - FSx for Lustre
  - FSx for NetApp ONTAP
  - FSx for OpenZFS

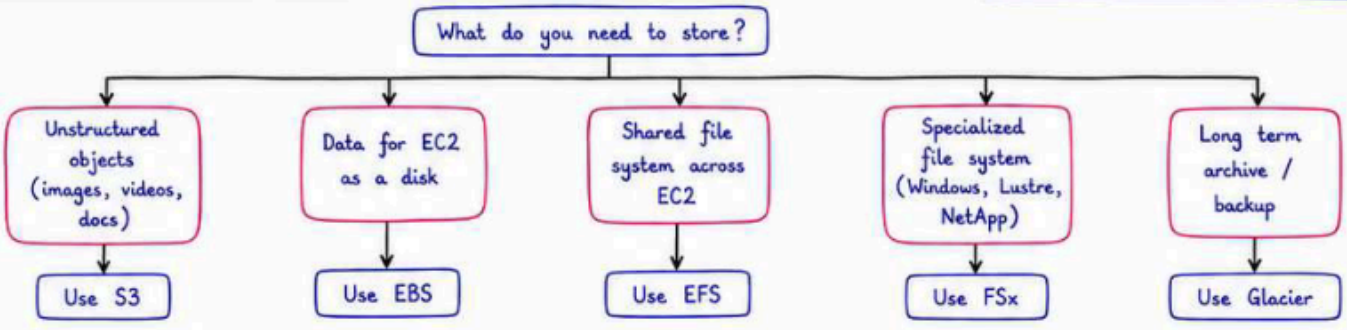
7. Glacier (Archive Storage)

- Low cost storage for long-term data archiving.
- Two retrieval options :
  - Flexible Retrieval (minutes / hours)
  - Deep Archive (hours / days)
- Use case : Compliance, backups, long term retention.

8. Storage Comparison

Service	Type	Use Case
S3	Object	Large scale, internet accessible data
EBS	Block	EC2 boot volume, databases
EFS	File	Shared file system across instances
FSx	File	Third-party file systems
Glacier	Archive	Long term backup, compliance

9. Storage Selection Guide



4. NETWORKING IN AWS

Build isolated networks and control traffic securely.

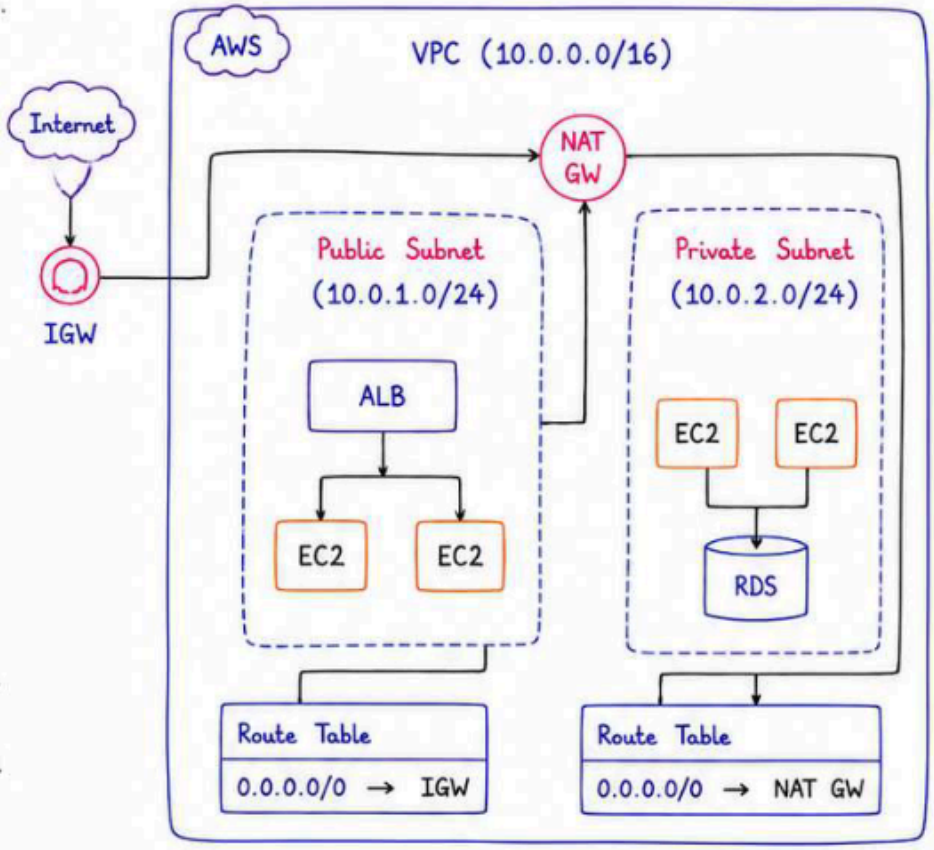
1. VPC (Virtual Private Cloud)

- A logically isolated virtual network in AWS.
- You have full control over IP ranges, subnets, route tables, gateways, etc.
- Each AWS account can create multiple VPCs.

2. Key Components

- **Subnet** : A range of IP addresses in a VPC.
- **Route Table** : Controls where network traffic is directed.
- **Internet Gateway (IGW)** : Allows communication between VPC and Internet.
- **NAT Gateway** : Allows private subnet instances to access Internet.
- **Security Group** : Acts as virtual firewall at instance level (stateful).
- **Network ACL (NACL)** : Acts as firewall at subnet level (stateless).

3. VPC Architecture (Example)



4. Security Group vs NACL

Feature	Security Group	Network ACL
Level	Instance Level	Subnet Level
Type	Stateful	Stateless
Rule	Allow Rules	Allow + Deny Rules
Default	All Deny	All Allow
Use Case	Control traffic to instances	Control traffic at subnet level

5. Connectivity Options

- **VPC Peering** : Connect two VPCs privately.
- **AWS Direct Connect** : Dedicated private network connection from on-premises to AWS.
- **VPN Connection** : Encrypted connection between on-premises and VPC.

6. IP Addressing in VPC

- **Default VPC CIDR** : 172.31.0.0/16
- You can define your own CIDR block.
- Each subnet must have unique CIDR range.

7. Common Ports (Security Groups)

Port	Protocol	Use
22	TCP	SSH (Linux)
80	TCP	HTTP
443	TCP	HTTPS
3306	TCP	MySQL
5432	TCP	PostgreSQL
3389	TCP	RDP (Windows)

☆ Key Takeaways

- ✓ VPC is the foundation of your cloud network.
- ✓ Use public subnets for resources that need Internet access.
- ✓ Use private subnets for secure backend resources.
- ✓ Control traffic using Security Groups and NACLs.
- ✓ Always follow least privilege access.

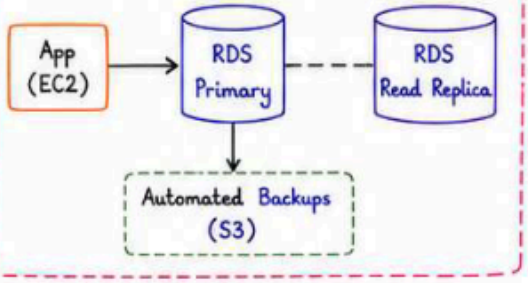
5. DATABASE SERVICES

Managed database services for all your application needs.

1. RDS (Relational Database Service)

- Managed relational database in the cloud.
- Supports MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Aurora.
- Features : Automated backups, patching, Multi-AZ, Read Replicas.
- Use case : Traditional relational applications.

RDS Architecture



2. Aurora

- AWS proprietary relational database.
- MySQL & PostgreSQL compatible.
- 5x performance of MySQL.
- Features : Auto scaling storage, Multi-AZ, Failover in seconds.
- Use case : High performance, high availability apps.

3. DynamoDB

- Fully managed NoSQL (Key-Value & Document DB).
- Serverless, auto scaling.
- Single-digit millisecond latency.
- Features : On-demand capacity, Global Tables, TTL, Streams.
- Use case : Real-time apps, session stores, IoT, gaming.

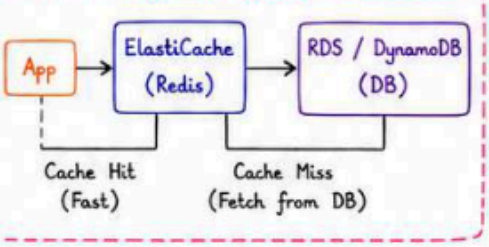
DynamoDB Table Structure

Partition Key (Hash Key)	Sort Key (Range Key)
UserID	Timestamp
UserID	OrderID
UserID	ProductID
...	...

4. ElastiCache

- In-memory caching service.
- Supports Redis & Memcached.
- Improves application performance and reduces DB load.
- Use case : Caching, session store, leaderboards, real-time analytics.

How it Works



5. Redshift

- Data warehouse service for analytics.
- Columnar storage.
- Petabyte scale.
- Integrates with S3, QuickSight, BI tools.
- Use case : Data warehousing, reporting, BI dashboards.

Redshift Flow



6. Database Comparison

Service	Type	Best For
RDS	Relational	Traditional apps, transactions
Aurora	Relational (High Perf.)	High performance, availability
DynamoDB	NoSQL	Real-time, scalable applications
ElastiCache	In-Memory Cache	Caching, session store, reducing DB load
Redshift	Data Warehouse	Analytics, reporting, BI

7. When to Use What?

- Need relational DB with ease of use? → Use RDS
- Need high performance & availability? → Use Aurora
- Need NoSQL for massive scale? → Use DynamoDB
- Need caching for faster performance? → Use ElastiCache

☆ Key Takeaways

- ✓ Choose the right database based on your application needs.
- ✓ RDS/Aurora for relational, DynamoDB for NoSQL.
- ✓ Use ElastiCache to improve performance.
- ✓ Use Redshift for large scale analytics and reporting.
- ✓ All services are fully managed and highly available.

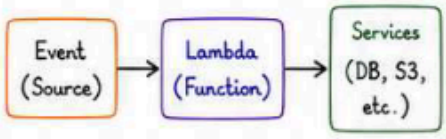
6. SERVERLESS & MESSAGING

Build scalable, event-driven applications without managing servers.

1. AWS Lambda

- Serverless compute service.
- Run code without provisioning or managing servers.
- Pay only for the time your code runs.
- Supports many runtimes: Python, Node.js, Java, Go, etc.
- Use case : File processing, real-time data processing, APIs, automation.

How Lambda Works



2. API Gateway

- Fully managed service to create, publish, maintain, monitor and secure APIs.
- Acts as a front door for your applications.
- Integrates easily with Lambda.
- Use case : REST APIs, microservices, mobile backends.

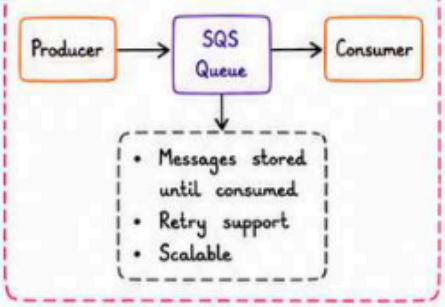
API Gateway Flow



3. SQS (Simple Queue Service)

- Fully managed message queue service.
- Decouples and scales microservices.
- Messages remain in the queue until processed.
- Types :
  - Standard Queue → At-least-once delivery, high throughput.
  - FIFO Queue → Exactly-once processing, ordered delivery.
- Use case : Background jobs, task queues, decoupling services.

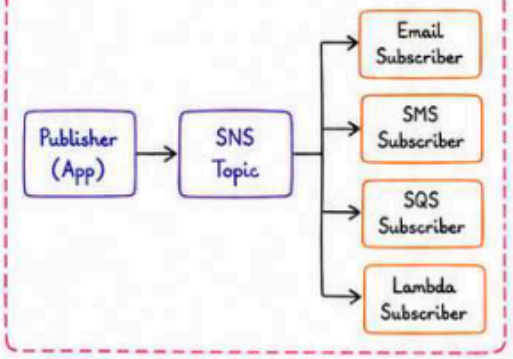
SQS - How it Works



4. SNS (Simple Notification Service)

- Fully managed pub/sub messaging service.
- Sends messages (notifications) to multiple subscribers.
- Supports Email, SMS, HTTP/S, Lambda, SQS.
- Use case : Alerts, notifications, fan-out messaging.

SNS - How it Works



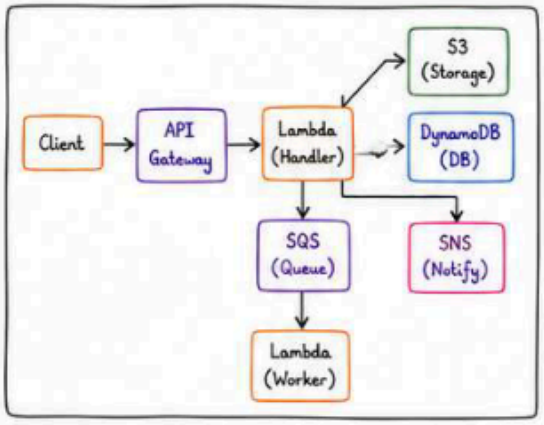
5. EventBridge (CloudWatch Events)

- Serverless event bus.
- Route events from AWS services, custom apps, SaaS apps.
- Trigger Lambda, Step Functions, SQS, SNS, etc.
- Use case : Schedule tasks, react to changes, automation.

6. Step Functions

- Orchestrates multiple AWS services into serverless workflows.
- Visual workflow.
- Handles errors, retries, timeouts.
- Use case : Complex workflows, approvals, ETL pipelines.

7. Serverless Architecture (Example)



8. Services Comparison

Service	Type	Best For	
Lambda	Compute	Run code	Pay per execution
API Gateway	API	Expose APIs	Secure & scalable APIs
SQS	Queue	Decouple services	Reliable message queues
SNS	Pub/Sub	Notifications	Fan-out messaging
EventBridge	Event Bus	Event routing	Schedule & react to events
Step Functions	Workflow	Orchestrate services	Visual workflows

☆ Key Takeaways

- ✓ Use Lambda for compute without servers.
- ✓ Use API Gateway to expose serverless APIs.
- ✓ Use SQS for decoupling and background jobs.
- ✓ Use SNS for notifications and pub/sub.
- ✓ Use EventBridge to route and schedule events.
- ✓ Use Step Functions for complex workflows.

7. CONTAINERS & ORCHESTRATION (ECS)

Package, deploy and manage applications at scale using containers.

1. What is a Container?

- A container packages an application with its code, runtime, libraries and dependencies.
- Lightweight, portable and consistent across environments.
- Runs using container engine (Docker).
- Shares host OS kernel.
- Use case : Microservices, CI/CD, portable apps.

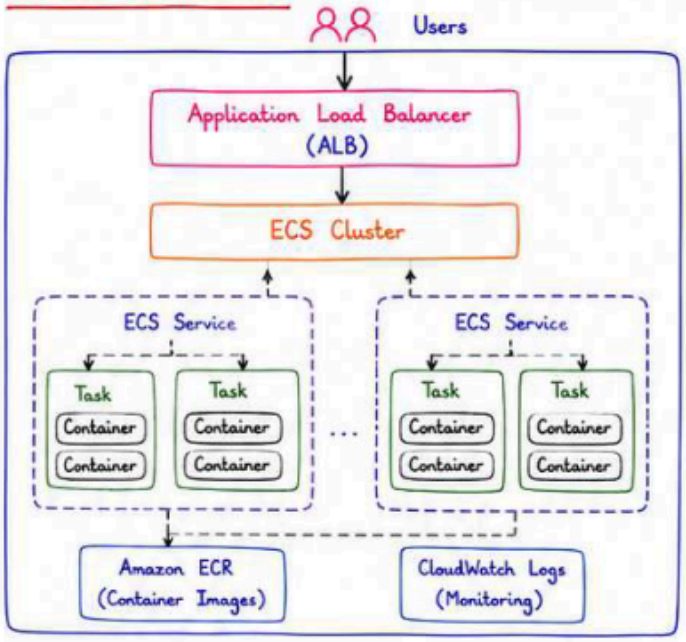
2. Docker Basics

- Build image → Run container
- Image is a read-only template.
- Container is a running instance of image.
- Key Commands :
  - `docker build -t myapp .`
  - `docker run -d -p 8080:8080 myapp`
  - `docker ps`
  - `docker logs <container_id>`
  - `docker stop <container_id>`

3. ECS (Elastic Container Service)

- Fully managed container orchestration service by AWS.
- No need to manage underlying infrastructure.
- Supports Docker containers.
- Integrates with ALB, IAM, CloudWatch, ECR, VPC, etc.
- Use case : Deploy and manage containers at scale.

4. ECS Architecture



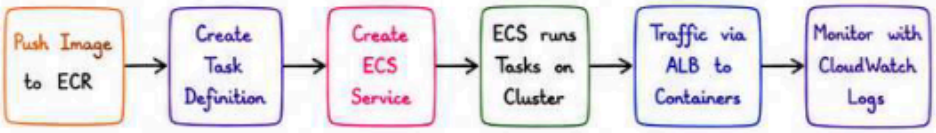
5. ECS Key Components

Component	Description
Cluster	Logical grouping of resources for running containers.
Service	Keeps desired number of tasks running and ensures high availability.
Task	Definition of one or more containers.
Task Definition	Blueprint for your application (CPU, Memory, Image, Port, etc.).
ECS Agent	Runs on EC2 instances and manages containers.
ECR	Private container registry to store Docker images.
ALB	Distributes traffic across tasks.

6. Launch Types

- EC2 Launch Type : You manage EC2 instances. More control, more responsibility.
- Fargate Launch Type : Serverless compute for containers. No server management.
- Use case : Fargate → Simplicity
  EC2 → More control, cost optimization

7. ECS Workflow



8. ECS Benefits

- ✓ No infrastructure management (with Fargate).
- ✓ Auto scaling & self healing.
- ✓ High availability across AZs.
- ✓ Deep integration with AWS services.
- ✓ Secure and isolated with IAM & VPC.
- ✓ Cost effective - pay for what you use.

9. Use Cases

- Microservices applications
- Web applications
- Backend APIs
- Batch processing jobs
- Event-driven applications

☆ Key Takeaways

- ✓ Containers make apps portable and consistent.
- ✓ ECS helps you run and scale containers without managing servers.
- ✓ Use Fargate for simplicity, EC2 for more control.
- ✓ Integrate with AWS services for monitoring, security and networking.

8. INFRASTRUCTURE AS CODE (IaC)

Manage and provision infrastructure using code for consistency and speed.

1. What is IaC?

- Infrastructure as Code (IaC) means defining and managing infrastructure using code instead of manual processes.
- Helps in automation, version control, reusability and consistency.
- You write code → Tools provision and manage the infrastructure.
- Use case : AWS, Azure, GCP resources provisioning, environments setup, scaling, networking, etc.

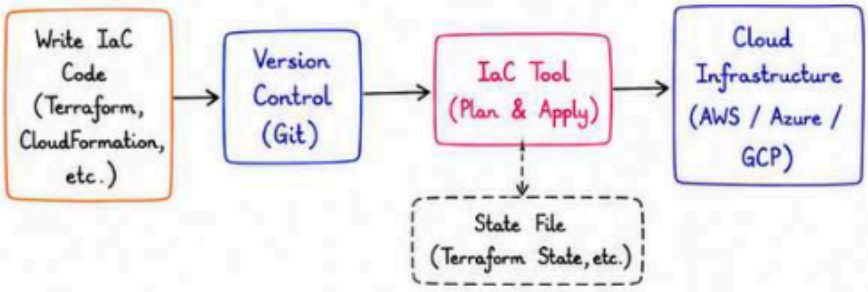
2. Popular IaC Tools

Tool	Type
Terraform	Multi-Cloud (AWS, Azure, GCP, etc.)
AWS CloudFormation	AWS Native
Azure Bicep	Azure Native
Pulumi	Multi-Language IaC
Ansible	Configuration Management
Chef / Puppet	Configuration Management

3. IaC Benefits

- ✓ Consistency across environments.
- ✓ Version control & collaboration.
- ✓ Automation & faster provisioning.
- ✓ Repeatable & reusable code.
- ✓ Drift detection & self-documenting.
- ✓ Cost & time effective.

4. How IaC Works



5. Terraform Basics

- Declarative language (HCL).
- Plan → Preview changes.
- Apply → Provision/Update resources.
- Manages state file to track resources.
- Supports multiple providers (AWS, Azure, GCP, etc.).

6. Terraform Workflow

1. Write configuration files (.tf)
2. Initialize Terraform
   `$ terraform init`
3. Validate configuration
   `$ terraform validate`
4. Preview changes
   `$ terraform plan`
5. Apply changes
   `$ terraform apply`
6. Destroy resources (if needed)
   `$ terraform destroy`

7. Terraform Example (AWS EC2)

```
# main.tf
provider "aws" {
  region = "ap-south-1"
}

resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"

  tags = {
    Name = "WebServer"
  }
}
```

8. State Management

- State file keeps track of real infrastructure.
- Stored locally or remotely (S3 + DynamoDB for locking).
- Enables Terraform to know what to create, update or delete.

9. Best Practices

- ✓ Keep modules reusable.
- ✓ Store sensitive data in Secrets Manager / Parameter Store.
- ✓ Use remote state for team collaboration.
- ✓ Enable versioning & state locking.
- ✓ Follow least privilege for cloud access.

☆ Key Takeaways

- ✓ IaC brings automation, consistency and reliability.
- ✓ Terraform is a powerful multi-cloud IaC tool.
- ✓ Always keep your IaC code in version control.
- ✓ Use remote state for better collaboration and safety.
- ✓ IaC is the foundation of modern DevOps.

9. MONITORING, LOGGING & ALERTING

Ensure system health, detect issues early and respond quickly.

1. Why Monitoring?

- Detect issues in real-time.
- Understand system performance.
- Improve reliability & availability.
- Reduce downtime and MTTR.
- Use case : Servers, databases, applications, infrastructure.

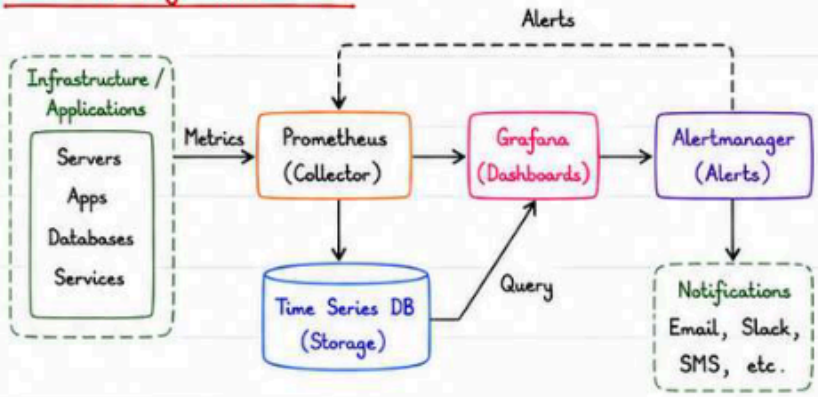
2. What to Monitor?

- Infrastructure : CPU, Memory, Disk, Network.
- Applications : Response time, Throughput, Error rate.
- Services : Uptime, Health checks.
- Logs : Errors, Exceptions.
- Business Metrics : Users, Orders, Revenue.

3. Monitoring Tools

Tool	Use
Prometheus	Metrics collection & alerting
Grafana	Visualization & dashboards
CloudWatch	AWS monitoring service
Datadog	Full-stack monitoring
New Relic	APM & infrastructure monitoring
Zabbix	Infrastructure monitoring

4. Monitoring Architecture



5. Logging Basics

- Logs record events, errors and system activities.
- Centralized logging helps in debugging & analysis.
- Use case : Application logs, access logs, audit logs.
- Best Practices :
  - Structured logs (JSON format)
  - Include timestamp, level, service, traceId
  - Avoid logging sensitive data
  - Log at correct levels (INFO, WARN, ERROR)

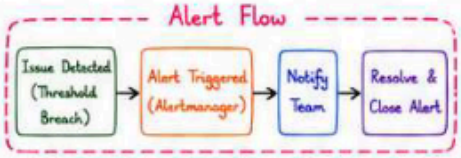
6. Logging Tools

Tool	Use
ELK Stack (Elasticsearch, Logstash, Kibana)	Log collection, search & visualization
Loki + Grafana	Log aggregation with Grafana
Graylog	Centralized log management
AWS CloudWatch Logs	Log collection & monitoring
Splunk	Log analytics & monitoring

7. Alerting Strategy

- Alert on issues, not on noise.
- Set appropriate thresholds.
- Use different channels for teams.
- Escalate if not acknowledged.
- Include runbook / troubleshooting link in alert.

Alert Flow



8. Common Metrics

- System : CPU %, Memory %, Disk I/O
- Network : Latency, Packet loss, Bandwidth
- Application : Request rate, Error rate, Response time
- Business : Signups, Orders, Payments
- Availability : Uptime, Downtime

9. Best Practices

- ✓ Monitor everything important.
- ✓ Set meaningful alerts with proper thresholds.
- ✓ Keep logs centralized and easily searchable.
- ✓ Create dashboards for key metrics.
- ✓ Review and improve alerts & dashboards regularly.
- ✓ Use runbooks to reduce MTTR.
- ✓ Follow "Observe → Analyze → Act → Improve" cycle.

☆ Key Takeaways

- ✓ Monitoring shows what is happening.
- ✓ Logging tells you why it happened.
- ✓ Alerting notifies you when action is needed.
- ✓ Good observability = Reliable systems.
- ✓ Plan, monitor, analyze and improve continuously.